OP-TEE 코드 흐름 상세 분석

이 문서는 OP-TEE의 주요 프로세스들을 코드 레벨에서 단계별로 추적하여 동작 과정을 자세히 설명합니다.

1. 시스템 부팅 과정 (Boot Process)

1.1 부팅 시퀀스 전체 흐름

```
Hardware Reset
↓
ARM Reset Vector (entry_a32.S)
↓
plat_cpu_reset_early()
↓
_start (entry_a32.S)
↓
boot_init_primary_early()
↓
boot_init_primary_late()
↓
boot_init_primary_runtime()
↓
```

```
init_tee_runtime()

↓

Ready for Service
```

1.2 ARM 어셈블리 엔트리 포인트 분석

파일: `core/arch/arm/kernel/entry_a32.S`

```
.section .identity_map, "ax"

.align 5

LOCAL_FUNC reset_vect_table , : , .identity_map

b . /* Reset */

b . /* Undef */

b . /* Syscall */

b . /* Prefetch abort */

b . /* Data abort */

b . /* Reserved */

b . /* IRQ */

b . /* FIQ */

END_FUNC reset_vect_table
```

동작 과정:

1. **하드웨어 리셋**: ARM 코어가 리셋되면 리셋 벡터로 점프
2. **Identity Mapping**: 초기 부팅 시 가상 주소 = 물리 주소

3. **벡터 테이블 설정**: 예외 처리를 위한 벡터 테이블 초기화

1.3 Primary CPU 초기화 (`boot_init_primary_early`)

파일: `core/arch/arm/kernel/boot.c`

```
static void init_primary(unsigned long pageable_part)
{
vaddr_t va = 0;

/*
* 1. 예외 마스킹 - 스레드 벡터로 전환하기 전에
* 비동기 예외를 마스크
*/
thread_set_exceptions(THREAD_EXCP_ALL);
/*
* 2. 타이머 주파수 저장 (멀티코어 동기화용)
*/
primary_save_cntfrq();
/*
* 3. VFP/SIMD 초기화
*/
init_vfp_sec();

/*
```

```
* 4. 암호화 확장 기능 검사
*/
if (IS_ENABLED(CFG_CRYPTO_WITH_CE))
check_crypto_extensions();


/*
* 5. AddressSanitizer 초기화
*/
init_asan();


/*
* 6. 메모리 할당자 초기화
*/
#ifdef CFG_WITH_PAGER
malloc_add_pool(__heap2_start, __heap2_end - __heap2_start);
#endif
#ifdef CFG_NS_VIRTUALIZATION
nex_malloc_add_pool(__nex_heap_start, __nex_heap_end - __nex_heap_start);
#else
malloc_add_pool(__heap1_start, __heap1_end - __heap1_start);
#endif


/*
```

```
* 7. MMU 및 메모리 맵 초기화
*/
core_mmu_save_mem_map();
core_mmu_init_phys_mem();
/*
* 8. 부팅 메모리 정리 및 해제
*/
boot_mem_foreach_padding(add_padding_to_pool, NULL);
va = boot_mem_release_unused();
/*
* 9. 페이저 런타임 초기화 (페이징 활성화된 경우)
*/
if (IS_ENABLED(CFG_WITH_PAGER)) {
thread_get_core_local()->curr_thread = 0;
init_pager_runtime(pageable_part);
}


/*
* 10. 스택 카나리 및 스레드별 초기화
*/
thread_init_canaries();
thread_init_per_cpu();
```

```
}
```

주요 단계별 설명:

1단계: 예외 처리 설정

```
thread_set_exceptions(THREAD_EXCP_ALL);
```

- 모든 예외를 마스크하여 초기화 중 인터럽트 방지
- 스레드 서브시스템이 완전히 초기화될 때까지 보호

2단계: VFP/SIMD 초기화

```
#ifdef ARM32
static void init_vfp_sec(void)
{
uint32_t cpacr = read_cpacr();
/* Advanced SIMD 기능 활성화 */
cpacr &= ~(CPACR_ASEDIS | CPACR_D32DIS);
/* CP10, CP11 (SIMD/VFP) 접근 권한 설정 */
cpacr |= CPACR_CP(10, CPACR_CP_ACCESS_FULL);
cpacr |= CPACR_CP(11, CPACR_CP_ACCESS_FULL);
write_cpacr(cpacr);
}
#endif
```

3단계: 암호화 확장 검사

```
static void check_crypto_extensions(void)
{
bool ce_supported = true;

if (!feat_aes_implemented() && IS_ENABLED(CFG_CRYPTO_AES_ARM_CE)) {
EMSG("AES instructions are not supported");
ce_supported = false;
}

if (!feat_sha1_implemented() && IS_ENABLED(CFG_CRYPTO_SHA1_ARM_CE)) {
EMSG("SHA1 instructions are not supported");
ce_supported = false;
}
/* ... 추가 검사 ... */
if (!ce_supported)
panic("HW doesn't support CE instructions");
}
```

1.4 메모리 발견 과정 (`discover_nsec_memory`)

파일: `core/kernel/boot.c`

```
void discover_nsec_memory(void)
{
struct core_mmu_phys_mem *mem;
const struct core_mmu_phys_mem *mem_begin = NULL;
const struct core_mmu_phys_mem *mem_end = NULL;
size_t nelems;
void *fdt = get_external_dt();


/*
* 1단계: 외부 Device Tree에서 메모리 정보 탐색
*/
if (fdt) {
mem = get_nsec_memory(fdt, &nelems);
if (mem) {
core_mmu_set_discovered_nsec_ddr(mem, nelems);
return;
}
DMSG("No non-secure memory found in external DT");
}


/*
* 2단계: 내장 Device Tree에서 메모리 정보 탐색
```

```
*/

fdt = get_embedded_dt();

if (fdt) {

mem = get_nsec_memory(fdt, &nelems);

if (mem) {

core_mmu_set_discovered_nsec_ddr(mem, nelems);

return;

}

DMSG("No non-secure memory found in embedded DT");

}



/*

* 3단계: 정적으로 등록된 메모리 사용

*/

mem_begin = phys_ddr_overall_begin;

mem_end = phys_ddr_overall_end;

nelems = mem_end - mem_begin;

if (nelems) {

assert(phys_ddr_overall_compat_begin == phys_ddr_overall_compat_end);

} else {

/* 호환성을 위한 deprecated API 사용 */

mem_begin = phys_ddr_overall_compat_begin;

mem_end = phys_ddr_overall_compat_end;
```

```
nelems = mem_end - mem_begin;

if (!nelems)

return;

DMSG("Warning register_dynamic_shm() is deprecated");

}



/*

* 4단계: 메모리 정보 복사 및 설정

*/

mem = nex_calloc(nelems, sizeof(*mem));

if (!mem)

panic();



memcpy(mem, phys_ddr_overall_begin, sizeof(*mem) * nelems);

core_mmu_set_discovered_nsec_ddr(mem, nelems);

}
```

Device Tree 파싱 과정:

```
static int get_nsec_memory_helper(void *fdt, struct core_mmu_phys_mem *mem)

{

const uint8_t *prop = NULL;

uint64_t a = 0, l = 0;

size_t prop_offs = 0, prop_len = 0;
```

```
int elems_total = 0;
int addr_size = 0, len_size = 0;
int offs = 0;

/* Device Tree의 주소/크기 셀 정보 획득 */
addr_size = fdt_address_cells(fdt, 0);
if (addr_size < 0) return 0;
len_size = fdt_size_cells(fdt, 0);
if (len_size < 0) return 0;

/* "memory" 타입의 노드들을 순회 */
while (true) {
offs = fdt_node_offset_by_prop_value(fdt, offs, "device_type",
"memory", sizeof("memory"));
if (offs < 0) break;

/* 노드 상태 확인 (secure/non-secure 접근 가능) */
if (fdt_get_status(fdt, offs) != (DT_STATUS_OK_NSEC | DT_STATUS_OK_SEC))
continue;

/* "reg" 속성에서 메모리 범위 정보 추출 */
prop = fdt_getprop(fdt, offs, "reg", &len);
if (!prop) continue;
```

```
prop_len = len;
for (n = 0, prop_offs = 0; prop_offs < prop_len; n++) {
/* 주소 추출 */
a = get_dt_val_and_advance(prop, &prop_offs, addr_size);
if (prop_offs >= prop_len) {
n--; break;
}

/* 크기 추출 */
l = get_dt_val_and_advance(prop, &prop_offs, len_size);
if (mem) {
mem->type = MEM_AREA_DDR_OVERALL;
mem->addr = a;
mem->size = l;
mem++;
}
}
elems_total += n;
}
return elems_total;
}
```

2. 스레드 관리 시스템 코드 흐름

2.1 스레드 초기화 과정

파일: `core/kernel/thread.c`

```
void thread_init_threads(size_t count)
{
size_t n = 0;

/*
* 1단계: 동적 설정인 경우 스레드 배열 할당
*/
if (IS_ENABLED(CFG_DYN_CONFIG)) {
assert(count <= CFG_NUM_THREADS);
threads = calloc(count, sizeof(*threads));
if (!threads)
panic();
thread_count = count;
} else {
assert(count == CFG_NUM_THREADS);
}

/*
```

```
* 2단계: 스레드별 스택 초기화
*/
init_thread_stacks();
print_stack_limits();
/*
* 3단계: 페이지 테이블 초기화
*/
pgt_init();


/*
* 4단계: 뮤텍스 락 의존성 추적 초기화
*/
mutex_lockdep_init();


/*
* 5단계: 각 스레드의 세션 스택 초기화
*/
for (n = 0; n < thread_count; n++)
TAILQ_INIT(&threads[n].tsd.sess_stack);
}
```

2.2 스택 초기화 상세 과정

```
#ifndef CFG_WITH_PAGER
static void init_thread_stacks(void)
{
vaddr_t va = 0;
size_t n = 0;

/* 각 스레드에 스택 할당 */
for (n = 0; n < thread_count; n++) {
if (IS_ENABLED(CFG_DYN_CONFIG))
va = alloc_stack(STACK_THREAD_SIZE, false);
else
va = GET_STACK_THREAD_BOTTOM(n);
threads[n].stack_va_end = va;
if (IS_ENABLED(CFG_WITH_STACK_CANARIES))
init_canaries(STACK_THREAD_SIZE, va);
}
}
#endif

/*
* 스택 할당 함수
*/
```

```
static vaddr_t alloc_stack(size_t stack_size, bool nex)

{

size_t l = stack_size_to_alloc_size(stack_size);

size_t rl = ROUNDUP(l, SMALL_PAGE_SIZE);

uint32_t flags = MAF_GUARD_HEAD; /* 가드 페이지 설정 */

vaddr_t end_va = 0;

vaddr_t va = 0;



if (nex)

flags |= MAF_NEX;

/* 가상 페이지 할당 */

va = virt_page_alloc(rl / SMALL_PAGE_SIZE, flags);

if (!va)

panic();



end_va = va + l - STACK_CANARY_SIZE / 2;

/* 스택 카나리 초기화 */

if (IS_ENABLED(CFG_WITH_STACK_CANARIES))

init_canaries(stack_size, end_va);



return end_va;

}
```

2.3 스택 보호 메커니즘

```
/*
* 스택 구조:
* [CANARY/2 | STACK_CHECK_EXTRA | STACK_SIZE | CANARY/2]
*/
static void init_canaries(size_t stack_size, vaddr_t va_end)
{
uint32_t *canary = NULL;

assert(va_end);
/* 시작 카나리 설정 */
canary = stack_end_va_to_start_canary(stack_size, va_end);
*canary = start_canary_value;
/* 끝 카나리 설정 */
canary = stack_end_va_to_end_canary(stack_size, va_end);
*canary = end_canary_value;
}

/*
* 스택 카나리 검사
*/
void thread_check_canaries(void)
```

```
{
vaddr_t va = 0;
size_t n = 0;

if (IS_ENABLED(CFG_WITH_STACK_CANARIES)) {
/* 코어별 스택 검사 */
for (n = 0; n < thread_core_count; n++) {
if (thread_core_local[n].tmp_stack_va_end) {
va = thread_core_local[n].tmp_stack_va_end + STACK_TMP_OFFS;
check_stack_canary("tmp_stack", n, STACK_TMP_SIZE, va);
}

va = thread_core_local[n].abt_stack_va_end;
if (va)
check_stack_canary("abt_stack", n, STACK_ABT_SIZE, va);
}
}

/* 스레드별 스택 검사 */
if (IS_ENABLED(CFG_WITH_STACK_CANARIES) &&
!IS_ENABLED(CFG_WITH_PAGER) &&
!IS_ENABLED(CFG_NS_VIRTUALIZATION)) {
for (n = 0; n < thread_count; n++) {
```

```
va = threads[n].stack_va_end;

if (va)

check_stack_canary("thread_stack", n, STACK_THREAD_SIZE, va);

}

}

}
```

3. 시스템 콜 처리 과정

3.1 시스템 콜 디스패치 메커니즘

파일: `core/kernel/scall.c`

```
/*

* 시스템 콜 테이블 - TEE_SCN_xxx 순서로 정렬

*/

static const struct syscall_entry tee_syscall_table[] = {

SYSCALL_ENTRY(syscall_sys_return), /* 0 */

SYSCALL_ENTRY(syscall_log), /* 1 */

SYSCALL_ENTRY(syscall_panic), /* 2 */

SYSCALL_ENTRY(syscall_get_property), /* 3 */

SYSCALL_ENTRY(syscall_get_property_name_to_index), /* 4 */

SYSCALL_ENTRY(syscall_open_ta_session), /* 5 */

SYSCALL_ENTRY(syscall_close_ta_session), /* 6 */
```

```
SYSCALL_ENTRY(syscall_invoke_ta_command), /* 7 */

SYSCALL_ENTRY(syscall_check_access_rights), /* 8 */

SYSCALL_ENTRY(syscall_get_cancellation_flag), /* 9 */

SYSCALL_ENTRY(syscall_unmask_cancellation), /* 10 */

SYSCALL_ENTRY(syscall_mask_cancellation), /* 11 */

SYSCALL_ENTRY(syscall_wait), /* 12 */

SYSCALL_ENTRY(syscall_get_time), /* 13 */

SYSCALL_ENTRY(syscall_set_ta_time), /* 14 */

/* 암호화 관련 시스템 콜들 */

SYSCALL_ENTRY(syscall_cryp_state_alloc), /* 15 */

SYSCALL_ENTRY(syscall_cryp_state_copy), /* 16 */

SYSCALL_ENTRY(syscall_cryp_state_free), /* 17 */

SYSCALL_ENTRY(syscall_hash_init), /* 18 */

SYSCALL_ENTRY(syscall_hash_update), /* 19 */

SYSCALL_ENTRY(syscall_hash_final), /* 20 */

/* ... 더 많은 시스템 콜들 ... */

};



/*

* 시스템 콜 함수 획득

*/

static syscall_t get_tee_syscall_func(size_t num)

{
```

```
struct syscall_entry *sc_table = (void *)tee_syscall_table;


static_assert(ARRAY_SIZE(tee_syscall_table) == (TEE_SCN_MAX + 1));


if (num > TEE_SCN_MAX)
return (syscall_t)syscall_not_supported;


/*
* Speculation barrier: 추측 실행 공격 방지
* 배열 경계를 벗어난 접근 방지
*/
speculation_barrier_load_check(num, TEE_SCN_MAX);


return sc_table[num].fn;
}
```

3.2 시스템 콜 실행 과정

```
/*
* 메인 시스템 콜 핸들러
*/
void tee_svc_handler(struct thread_svc_regs *regs)
{
```

```
size_t num = regs->r7; /* ARM에서 시스템 콜 번호는 r7 레지스터 */

syscall_t scf = get_tee_syscall_func(num);


/* 시스템 콜 추적 */

trace_syscall(num);

ftrace_syscall_enter(num);


/* 실제 시스템 콜 실행 */

regs->r0 = scf(regs->r0, regs->r1, regs->r2,

regs->r3, regs->r4, regs->r5);


ftrace_syscall_leave();

}


/*

* LDELF (동적 링커) 시스템 콜 핸들러

*/

void ldelf_svc_handler(struct thread_svc_regs *regs)

{

size_t num = regs->r7;

syscall_t scf = get_ldelf_syscall_func(num);


trace_syscall(num);
```

```
ftrace_syscall_enter(num);


regs->r0 = scf(regs->r0, regs->r1, regs->r2,
regs->r3, regs->r4, regs->r5);


ftrace_syscall_leave();
}
```

3.3 시스템 콜 예제: 암호화 상태 할당

```
/*
* syscall_cryp_state_alloc 구현
*/
TEE_Result syscall_cryp_state_alloc(unsigned long algo, unsigned long mode,
unsigned long key1, unsigned long key2)
{
TEE_Result res = TEE_SUCCESS;
struct tee_cryp_state *cs = NULL;
struct ts_session *sess = NULL;


/*
* 1단계: 현재 세션 획득
*/
```

```
sess = ts_get_current_session();


/*
* 2단계: 암호화 상태 구조체 할당
*/
cs = calloc(1, sizeof(struct tee_cryp_state));
if (!cs)
return TEE_ERROR_OUT_OF_MEMORY;


/*
* 3단계: 암호화 상태 초기화
*/
cs->algo = algo;
cs->mode = mode;
cs->key1 = key1;
cs->key2 = key2;
cs->state = CRYP_STATE_UNINITIALIZED;


/*
* 4단계: 세션의 암호화 상태 리스트에 추가
*/
TAILQ_INSERT_TAIL(&sess->cryp_states, cs, link);
```

```
/*
* 5단계: 핸들 반환 (포인터를 핸들로 사용)
*/
return (TEE_Result)cs;
}
```

4. TA 관리 시스템 코드 흐름

4.1 TA 세션 열기 과정

파일: `core/kernel/tee_ta_manager.c`

```
TEE_Result tee_ta_open_session(TEE_ErrorOrigin *err,
struct tee_ta_session **sess,
struct tee_ta_session_head *open_sessions,
const TEE_UUID *uuid,
const TEE_Identity *clnt_id,
uint32_t cancel_req_to,
struct tee_ta_param *param)
{
TEE_Result res = TEE_SUCCESS;
struct tee_ta_session *s = NULL;
struct tee_ta_ctx *ctx = NULL;
struct ts_ctx *ts_ctx = NULL;
```

```
bool panicked = false;

bool was_busy = false;



/*

* 1단계: 세션 초기화

*/

res = tee_ta_init_session(err, open_sessions, uuid, &s);

if (res != TEE_SUCCESS) {

DMSG("init session failed 0x%x", res);

return res;

}



/*

* 2단계: 매개변수 검증

*/

if (!check_params(s, param))

return TEE_ERROR_BAD_PARAMETERS;



ts_ctx = s->ts_sess.ctx;

ctx = ts_to_ta_ctx(ts_ctx);



/*

* 3단계: TA 컨텍스트를 busy 상태로 설정 시도
```

```
*/
if (tee_ta_try_set_busy(ctx)) {
if (!ctx->panicked) {
/* 클라이언트 신원 저장 */
s->clnt_id = *clnt_id;
s->param = param;
set_invoke_timeout(s, cancel_req_to);
/*
* 4단계: TA의 OpenSession 엔트리 포인트 호출
*/
res = ts_ctx->ops->enter_open_session(&s->ts_sess);
s->param = NULL;
}

panicked = ctx->panicked;
if (panicked) {
maybe_release_ta_ctx(ctx);
res = TEE_ERROR_TARGET_DEAD;
}

tee_ta_clear_busy(ctx);
} else {
/* 데드락 방지 */
```

```
res = TEE_ERROR_BUSY;

was_busy = true;

}



/*

* 5단계: 에러 원인 설정

*/

if (panicked || was_busy)

*err = TEE_ORIGIN_TEE;

else

*err = s->err_origin;



/*

* 6단계: 세션 참조 해제 및 정리

*/

tee_ta_put_session(s);

if (panicked || res != TEE_SUCCESS)

tee_ta_close_session(s, open_sessions, KERN_IDENTITY);



if (!res)

*sess = s;

else

EMSG("Failed for TA %pUl. Return error %#"PRIx32, uuid, res);
```

```
return res;
}
```

4.2 TA 세션 초기화 상세 과정

```
static TEE_Result tee_ta_init_session(TEE_ErrorOrigin *err,
struct tee_ta_session_head *open_sessions,
const TEE_UUID *uuid,
struct tee_ta_session **sess)
{
TEE_Result res;
struct tee_ta_session *s = calloc(1, sizeof(struct tee_ta_session));

*err = TEE_ORIGIN_TEE;
if (!s)
return TEE_ERROR_OUT_OF_MEMORY;

/*
* 1단계: 세션 기본 설정
*/
s->cancel_mask = true;
condvar_init(&s->refc_cv);
```

```
condvar_init(&s->lock_cv);

s->lock_thread = THREAD_ID_INVALID;

s->ref_count = 1;



mutex_lock(&tee_ta_mutex);

/*

* 2단계: 고유한 세션 ID 생성

*/

s->id = new_session_id(open_sessions);

if (!s->id) {

res = TEE_ERROR_OVERFLOW;

goto err_mutex_unlock;

}



TAILQ_INSERT_TAIL(open_sessions, s, link);



/*

* 3단계: 기존 로드된 TA 검색

*/

res = tee_ta_init_session_with_context(s, uuid);

if (res == TEE_SUCCESS || res != TEE_ERROR_ITEM_NOT_FOUND) {

mutex_unlock(&tee_ta_mutex);

goto out;
```

```
}

/*
* 4단계: Secure Partition 검색
*/
res = stmm_init_session(uuid, s);
if (res == TEE_SUCCESS || res != TEE_ERROR_ITEM_NOT_FOUND) {
mutex_unlock(&tee_ta_mutex);
if (res == TEE_SUCCESS)
res = stmm_complete_session(s);
goto out;
}

/*
* 5단계: Pseudo TA 검색
*/
res = tee_ta_init_pseudo_ta_session(uuid, s);
if (res == TEE_SUCCESS || res != TEE_ERROR_ITEM_NOT_FOUND) {
mutex_unlock(&tee_ta_mutex);
goto out;
}

/*
```

```
* 6단계: User TA 검색 및 로드

*/

res = tee_ta_init_user_ta_session(uuid, s);

mutex_unlock(&tee_ta_mutex);

if (res == TEE_SUCCESS)

res = tee_ta_complete_user_ta_session(s);



out:

if (!res) {

*sess = s;

return TEE_SUCCESS;

}



mutex_lock(&tee_ta_mutex);

TAILQ_REMOVE(open_sessions, s, link);

err_mutex_unlock:

mutex_unlock(&tee_ta_mutex);

free(s);

return res;

}
```

4.3 단일 인스턴스 TA 동기화

```
static bool tee_ta_try_set_busy(struct tee_ta_ctx *ctx)
{
bool rc = true;

/*
* 동시 실행 가능한 TA는 항상 성공
*/
if (ctx->flags & TA_FLAG_CONCURRENT)
return true;

mutex_lock(&tee_ta_mutex);

/*
* 단일 인스턴스 TA 처리
*/
if (ctx->flags & TA_FLAG_SINGLE_INSTANCE)
lock_single_instance();

if (has_single_instance_lock()) {
if (ctx->busy) {
/*
* 단일 인스턴스 락을 보유하고 있지만 TA가 busy 상태
```

```
* 데드락 방지를 위해 락 해제 후 false 반환

*/

rc = false;

if (ctx->flags & TA_FLAG_SINGLE_INSTANCE)

unlock_single_instance();

}

} else {

/*

* 단일 인스턴스 락을 보유하지 않음

* TA가 사용 가능해질 때까지 대기 가능

*/

while (ctx->busy)

condvar_wait(&ctx->busy_cv, &tee_ta_mutex);

}



/* busy 상태로 설정 */

ctx->busy = true;



mutex_unlock(&tee_ta_mutex);

return rc;

}



static void lock_single_instance(void)
```

```
{
/* tee_ta_mutex가 이미 잠겨있어야 함 */
if (tee_ta_single_instance_thread != thread_get_id()) {
/* 단일 인스턴스 락이 사용 가능할 때까지 대기 */
while (tee_ta_single_instance_thread != THREAD_ID_INVALID)
condvar_wait(&tee_ta_cv, &tee_ta_mutex);

tee_ta_single_instance_thread = thread_get_id();
assert(tee_ta_single_instance_count == 0);
}

tee_ta_single_instance_count++;
}
```

5. 메모리 관리 시스템 코드 흐름

5.1 MMU 초기화 과정

파일: `core/mm/core_mmu.c`

```
void core_mmu_init_phys_mem(void)
{
struct memory_map *mem_map = get_memory_map();
size_t n = 0;
```

```
/*
* 1단계: 플랫폼별 메모리 영역 등록
*/
for (n = 0; n < ARRAY_SIZE(secure_only); n++) {
if (!secure_only[n].size)
continue;
add_phys_mem(mem_map, "secure_only", MEM_AREA_SEC_RAM_OVERALL,
secure_only[n].paddr, secure_only[n].size);
}


/*
* 2단계: Non-secure 공유 메모리 영역 등록
*/
for (n = 0; n < ARRAY_SIZE(nsec_shared); n++) {
if (!nsec_shared[n].size)
continue;
add_phys_mem(mem_map, "nsec_shared", MEM_AREA_NSEC_SHM,
nsec_shared[n].paddr, nsec_shared[n].size);
}


/*
* 3단계: 등록된 물리 메모리 검증
```

```
*/
verify_special_mem_areas(mem_map, phys_sdp_mem_begin, phys_sdp_mem_end,
"SDP memory");


/*
* 4단계: 가상 주소 공간 할당
*/
add_va_space(mem_map, MEM_AREA_RES_VASPACE, CFG_RESERVED_VASPACE_SIZE);
add_va_space(mem_map, MEM_AREA_SHM_VASPACE, SHM_VASPACE_SIZE);


/*
* 5단계: 메모리 맵 정렬 및 최적화
*/
qsort(mem_map->map, mem_map->count, sizeof(*mem_map->map),
cmp_mmap_by_lower_va);


dump_mmap_table(mem_map);
}
```

5.2 페이지 할당자 동작

```
/*
* 페이지 할당 함수
```

```
*/
void *page_alloc(size_t size, enum page_alloc_type type)
{
struct page_alloc_ctx *ctx = get_page_alloc_ctx(type);
void *ptr = NULL;
size_t aligned_size = ROUNDUP(size, SMALL_PAGE_SIZE);


mutex_lock(&ctx->mutex);


/*
* 1단계: 메모리 풀에서 할당 시도
*/
ptr = bget_alloc(ctx->pool, aligned_size);
if (!ptr) {
/*
* 2단계: 풀 확장 시도
*/
if (expand_pool(ctx, aligned_size)) {
ptr = bget_alloc(ctx->pool, aligned_size);
}
}


if (ptr) {
```

```
/*
* 3단계: 할당된 메모리 초기화
*/
memset(ptr, 0, aligned_size);
/*
* 4단계: 메모리 태깅 (MTE 지원 시)
*/
if (IS_ENABLED(CFG_MEMTAG))
ptr = memtag_set_random_tags(ptr, aligned_size);
}


mutex_unlock(&ctx->mutex);
return ptr;
}


/*
* 페이지 해제 함수
*/
void page_free(void *ptr, size_t size)
{
struct page_alloc_ctx *ctx = NULL;
size_t aligned_size = ROUNDUP(size, SMALL_PAGE_SIZE);
```

```
if (!ptr)
return;

/*
* 1단계: 할당 컨텍스트 찾기
*/
ctx = find_page_alloc_ctx(ptr);
if (!ctx) {
EMSG("Invalid pointer %p", ptr);
return;
}

mutex_lock(&ctx->mutex);

/*
* 2단계: 메모리 지우기 (보안)
*/
memzero_explicit(ptr, aligned_size);

/*
* 3단계: 메모리 풀에 반환
*/
bget_free(ctx->pool, ptr);
```

```
mutex_unlock(&ctx->mutex);

}
```

6. 암호화 서비스 코드 흐름

6.1 암호화 연산 초기화

파일: `core/tee/tee_svc_cryp.c`

```
TEE_Result syscall_cipher_init(unsigned long state, unsigned long iv,

size_t iv_len)

{

TEE_Result res = TEE_SUCCESS;

struct tee_cryp_state *cs = NULL;

struct tee_ta_session *sess = NULL;

void *iv_buf = NULL;



/*

* 1단계: 현재 세션 및 암호화 상태 획득

*/

res = tee_ta_get_current_session(&sess);

if (res != TEE_SUCCESS)

return res;
```

```
res = tee_svc_cryp_get_state(sess, tee_svc_uref_to_vaddr(state), &cs);
if (res != TEE_SUCCESS)
return res;


/*
* 2단계: IV(Initialization Vector) 복사
*/
if (iv_len > 0) {
res = tee_mmu_check_access_rights(to_user_ta_ctx(sess->ctx),
TEE_MEMORY_ACCESS_READ |
TEE_MEMORY_ACCESS_ANY_OWNER,
(uaddr_t)iv, iv_len);
if (res != TEE_SUCCESS)
goto out;


iv_buf = malloc(iv_len);
if (!iv_buf) {
res = TEE_ERROR_OUT_OF_MEMORY;
goto out;
}


res = copy_from_user_private(iv_buf, (void *)iv, iv_len);
```

```
if (res != TEE_SUCCESS)
goto out;
}


/*
* 3단계: 암호화 엔진 초기화
*/
switch (TEE_ALG_GET_CLASS(cs->algo)) {
case TEE_OPERATION_CIPHER:
res = crypto_cipher_init(cs->ctx, cs->algo, cs->mode,
(uint8_t *)(cs->key1), cs->key1_len,
(uint8_t *)(cs->key2), cs->key2_len,
iv_buf, iv_len);
break;
case TEE_OPERATION_AE:
res = crypto_authenc_init(cs->ctx, cs->algo, cs->mode,
(uint8_t *)(cs->key1), cs->key1_len,
(uint8_t *)(cs->key2), cs->key2_len,
iv_buf, iv_len, 0, 0, 0);
break;
default:
res = TEE_ERROR_BAD_PARAMETERS;
break;
```

```
}

if (res == TEE_SUCCESS)
cs->state = CRYP_STATE_INITIALIZED;

out:
if (iv_buf)
free(iv_buf);
return res;
}
```

6.2 하드웨어 가속 활용

```
/*
* AES 하드웨어 가속 초기화
*/
static TEE_Result aes_hw_init(struct crypto_cipher_ctx *ctx,
TEE_OperationMode mode,
const uint8_t *key1, size_t key1_len,
const uint8_t *key2 __unused,
size_t key2_len __unused,
const uint8_t *iv, size_t iv_len)
{
```

```
struct aes_hw_ctx *c = to_aes_hw_ctx(ctx);

TEE_Result res = TEE_SUCCESS;



/*

* 1단계: 키 길이 검증

*/

if (key1_len != 16 && key1_len != 24 && key1_len != 32)

return TEE_ERROR_BAD_PARAMETERS;



/*

* 2단계: 하드웨어 엔진 초기화

*/

res = hw_aes_init(&c->hw_ctx, mode, key1, key1_len);

if (res != TEE_SUCCESS)

return res;



/*

* 3단계: IV 설정 (필요한 경우)

*/

if (iv && iv_len) {

if (iv_len != AES_BLOCK_SIZE)

return TEE_ERROR_BAD_PARAMETERS;

memcpy(c->iv, iv, iv_len);
```

```
c->iv_set = true;
}

/*
* 4단계: 컨텍스트 상태 설정
*/
c->mode = mode;
c->key_len = key1_len;
c->initialized = true;

return TEE_SUCCESS;
}

/*
* AES 블록 암호화 (하드웨어 가속)
*/
static TEE_Result aes_hw_update(struct crypto_cipher_ctx *ctx,
bool last_block,
const uint8_t *data, size_t len,
uint8_t *dst)
{
struct aes_hw_ctx *c = to_aes_hw_ctx(ctx);
TEE_Result res = TEE_SUCCESS;
```

```
size_t blocks = len / AES_BLOCK_SIZE;

size_t i = 0;



if (!c->initialized)

return TEE_ERROR_BAD_STATE;



/*

* 블록 단위로 처리

*/

for (i = 0; i < blocks; i++) {

const uint8_t *src = data + (i * AES_BLOCK_SIZE);

uint8_t *out = dst + (i * AES_BLOCK_SIZE);



switch (c->mode) {

case TEE_MODE_ENCRYPT:

res = hw_aes_encrypt_block(&c->hw_ctx, src, out);

break;

case TEE_MODE_DECRYPT:

res = hw_aes_decrypt_block(&c->hw_ctx, src, out);

break;

default:

res = TEE_ERROR_BAD_PARAMETERS;

break;
```

```
}

if (res != TEE_SUCCESS)
break;
}

return res;
}
```

이 문서는 OP-TEE의 주요 프로세스들을 코드 레벨에서 단계별로 추적하여 실제 동작 과정을 상세히 설명합니다. 각 함수의 호출 순서와 데이터 흐름을 따라가면서 시스템의 전체적인 동작을 이해할 수 있도록 구성했습니다.

7. 인터럽트 및 예외 처리 코드 흐름

7.1 예외 벡터 테이블 상세 분석

파일: `core/arch/arm/kernel/thread_a32.S`

```
/*
* 예외 벡터 테이블 - 각 예외 타입별 핸들러
*/
LOCAL_FUNC thread_vect_table , :
vector_std_smc_entry:
b thread_std_smc_entry /* SMC 호출 */
vector_fast_smc_entry:
```

```
b thread_fast_smc_entry /* Fast SMC 호출 */

vector_cpu_on_entry:

b thread_cpu_on_entry /* CPU 활성화 */

vector_cpu_off_entry:

b thread_cpu_off_entry /* CPU 비활성화 */

vector_cpu_suspend_entry:

b thread_cpu_suspend_entry /* CPU 일시정지 */

vector_cpu_resume_entry:

b thread_cpu_resume_entry /* CPU 재개 */

vector_system_off_entry:

b thread_system_off_entry /* 시스템 종료 */

vector_system_reset_entry:

b thread_system_reset_entry /* 시스템 리셋 */

END_FUNC thread_vect_table



/*

* 표준 SMC 엔트리 포인트

*/

LOCAL_FUNC thread_std_smc_entry , :

/* 1단계: 레지스터 상태 저장 */

push {r0-r7}

mov r0, sp /* 스택 포인터를 첫 번째 인자로 */

/* 2단계: 스레드 컨텍스트 전환 */
```

```
bl thread_handle_std_smc /* C 함수 호출 */

/* 3단계: 반환값 처리 */

cmp r0, #THREAD_SMC_RETURN_UNKNOWN_FUNCTION

beq .Lsmc_unknown

/* 4단계: 레지스터 복원 및 반환 */

pop {r0-r7}

smc #0 /* Secure Monitor로 반환 */

.Lsmc_unknown:

/* 알 수 없는 SMC 함수 처리 */

ldr r0, =TEE_ERROR_NOT_SUPPORTED

pop {r1-r7} /* r0는 에러 코드로 유지 */

smc #0

END_FUNC thread_std_smc_entry
```

7.2 SMC 핸들러 상세 구현

파일: `core/kernel/thread.c`

```
/*

* 표준 SMC 핸들러 - 모든 TEE 서비스 요청 처리

*/

uint32_t thread_handle_std_smc(uint32_t a0, uint32_t a1, uint32_t a2,

uint32_t a3, uint32_t a4, uint32_t a5,

uint32_t a6, uint32_t a7)
```

```
{
struct thread_smc_args args = {
.a0 = a0, .a1 = a1, .a2 = a2, .a3 = a3,
.a4 = a4, .a5 = a5, .a6 = a6, .a7 = a7
};
struct thread_core_local *l = thread_get_core_local();
uint32_t rv = 0;


/*
* 1단계: 현재 스레드 상태 확인
*/
if (l->curr_thread == THREAD_ID_INVALID) {
/*
* 새로운 스레드 할당 필요
*/
rv = thread_alloc_and_run(&args);
if (rv != 0) {
args.a0 = OPTEE_SMC_RETURN_ETHREAD_LIMIT;
goto out;
}
} else {
/*
* 기존 스레드 재개
```

```
*/
struct thread_ctx *thr = threads + l->curr_thread;
/*
* 2단계: 스레드 상태 검증
*/
assert(thr->state == THREAD_STATE_SUSPENDED);
/*
* 3단계: 레지스터 상태 복원
*/
copy_a0_to_a3(&thr->regs, &args);
/*
* 4단계: 스레드 실행 재개
*/
l->curr_thread = THREAD_ID_INVALID;
rv = thread_resume(&thr->regs);
/*
* 5단계: 실행 결과 처리
*/
if (rv == 0) {
copy_a0_to_a7(&args, &thr->regs);
} else {
args.a0 = OPTEE_SMC_RETURN_ERESUME;
}
```

```
}


out:
/*
* 6단계: 결과를 레지스터에 복사
*/
thread_copy_args_to_regs(&args);
return rv;
}


/*
* 새 스레드 할당 및 실행
*/
static uint32_t thread_alloc_and_run(struct thread_smc_args *args)
{
size_t n = 0;
struct thread_core_local *l = thread_get_core_local();
bool found_thread = false;


/*
* 1단계: 사용 가능한 스레드 검색
*/
for (n = 0; n < thread_count; n++) {
```

```
if (threads[n].state == THREAD_STATE_FREE) {
threads[n].state = THREAD_STATE_ACTIVE;
found_thread = true;
break;
}
}

if (!found_thread) {
return 1; /* 스레드 한계 초과 */
}

/*
* 2단계: 스레드 컨텍스트 초기화
*/
l->curr_thread = n;
threads[n].flags = 0;
threads[n].regs.pc = (uint32_t)thread_std_smc_entry;
threads[n].regs.cpsr = read_cpsr() & ARM32_CPSR_E;
/*
* 3단계: 스택 포인터 설정
*/
threads[n].regs.svc_sp = threads[n].stack_va_end;
/*
```

```
* 4단계: 인자 레지스터 설정

*/

copy_a0_to_a7(&threads[n].regs, args);

/*

* 5단계: 스레드 실행 시작

*/

return thread_enter_user_mode(&threads[n].regs);

}
```

7.3 인터럽트 처리 메커니즘

```
/*

* IRQ 인터럽트 핸들러

*/

void thread_irq_handler(void)

{

struct thread_core_local *l = thread_get_core_local();

uint32_t irq_num = 0;



/*

* 1단계: 인터럽트 번호 획득

*/

irq_num = gic_read_iar();
```

```
if (irq_num >= 1020) {

/* Spurious interrupt */

return;

}



/*

* 2단계: 현재 스레드 상태 저장

*/

if (l->curr_thread != THREAD_ID_INVALID) {

struct thread_ctx *thr = threads + l->curr_thread;

/* 스레드를 일시정지 상태로 변경 */

thr->state = THREAD_STATE_SUSPENDED_IRQ;

/* 컨텍스트 저장 */

thread_save_context(&thr->regs);

}



/*

* 3단계: 인터럽트 핸들러 호출

*/

itr_handle(irq_num);



/*

* 4단계: 인터럽트 완료 신호
```

```
*/
gic_write_eoir(irq_num);


/*
* 5단계: 스레드 재개 (필요한 경우)
*/
if (l->curr_thread != THREAD_ID_INVALID) {
struct thread_ctx *thr = threads + l->curr_thread;
if (thr->state == THREAD_STATE_SUSPENDED_IRQ) {
thr->state = THREAD_STATE_ACTIVE;
thread_restore_context(&thr->regs);
}
}
}


/*
* 개별 인터럽트 핸들러 등록 및 처리
*/
static struct itr_handler {
itr_handler_t handler;
uint32_t flags;
void *data;
} itr_handlers[MAX_INTR_NUM];
```

```
void itr_handle(size_t it)
{
struct itr_handler *h = NULL;

if (it >= MAX_INTR_NUM)
return;

h = itr_handlers + it;
if (h->handler) {
/*
* 등록된 핸들러 실행
*/
h->handler(it, h->data);
} else {
/*
* 등록되지 않은 인터럽트 - 마스크 처리
*/
DMSG("Disabling unhandled interrupt %zu", it);
itr_disable(it);
}
}
```

8. 동적 메모리 할당 상세 분석

8.1 Malloc 구현 세부사항

파일: `lib/libutils/ext/malloc.c`

```
/*
* 메모리 풀 구조체
*/
struct malloc_pool {
void *buf; /* 메모리 풀 시작 주소 */
size_t len; /* 풀 크기 */
bool used; /* 사용 중 여부 */
struct malloc_pool *next; /* 다음 풀 */
};

static struct malloc_pool *pool_list = NULL;
static struct mutex malloc_mu = MUTEX_INITIALIZER;

/*
* 메모리 할당 함수
*/
void *malloc(size_t size)
{
```

```
void *ptr = NULL;

struct malloc_pool *p = NULL;



if (!size)

return NULL;



/*

* 1단계: 크기 정렬 (최소 할당 단위로)

*/

size = ROUNDUP(size, MALLOC_ALIGNMENT);



mutex_lock(&malloc_mu);



/*

* 2단계: 적절한 풀에서 메모리 할당 시도

*/

for (p = pool_list; p; p = p->next) {

if (!p->used) {

ptr = bget_alloc(p->buf, size);

if (ptr) {

p->used = true;

break;

}
```

```
}
}

/*
* 3단계: 할당 실패 시 새 풀 생성 시도
*/
if (!ptr) {
ptr = try_expand_heap(size);
}

mutex_unlock(&malloc_mu);

/*
* 4단계: 할당된 메모리 초기화
*/
if (ptr) {
memset(ptr, 0, size);
/*
* 5단계: 메모리 태깅 (ARM MTE 지원 시)
*/
if (IS_ENABLED(CFG_MEMTAG)) {
ptr = memtag_set_random_tags(ptr, size);
}
```

```
}

return ptr;
}

/*
* 메모리 해제 함수
*/
void free(void *ptr)
{
struct malloc_pool *p = NULL;

if (!ptr)
return;

mutex_lock(&malloc_mu);

/*
* 1단계: 포인터가 속한 풀 찾기
*/
for (p = pool_list; p; p = p->next) {
if (ptr >= p->buf && ptr < (char *)p->buf + p->len) {
/*
```

```
* 2단계: 메모리 내용 지우기 (보안)

*/

size_t size = bget_size(ptr);

memzero_explicit(ptr, size);

/*

* 3단계: 풀에 메모리 반환

*/

bget_free(p->buf, ptr);

p->used = false;

break;

}

}


mutex_unlock(&malloc_mu);

}
```

8.2 BGET 할당자 상세 구현

```
/*

* BGET 블록 헤더 구조체

*/

struct bget_header {

size_t size; /* 블록 크기 */
```

```
struct bget_header *next; /* 다음 자유 블록 */

struct bget_header *prev; /* 이전 자유 블록 */

uint32_t magic; /* 매직 넘버 (무결성 검사) */

};


#define BGET_MAGIC_FREE 0xDEADBEEF

#define BGET_MAGIC_USED 0xCAFEBABE


/*

* 메모리 블록 할당

*/

void *bget_alloc(void *pool, size_t size)

{

struct bget_header *h = NULL;

struct bget_header *best = NULL;

size_t best_size = SIZE_MAX;

size_t total_size = 0;


if (!pool || !size)

return NULL;


/*

* 1단계: 헤더 크기 포함하여 총 크기 계산
```

```
*/

total_size = size + sizeof(struct bget_header);

total_size = ROUNDUP(total_size, BGET_ALIGNMENT);



/*

* 2단계: Best-fit 알고리즘으로 적절한 블록 찾기

*/

h = (struct bget_header *)pool;

while (h) {

/* 매직 넘버 검증 */

if (h->magic != BGET_MAGIC_FREE) {

EMSG("Corrupted free block at %p", h);

return NULL;

}



/* 크기가 적절하고 현재 최적보다 작은 블록 */

if (h->size >= total_size && h->size < best_size) {

best = h;

best_size = h->size;

/* 정확히 맞는 크기면 즉시 사용 */

if (h->size == total_size)

break;

}
```

```
h = h->next;

}


if (!best)

return NULL;


/*

* 3단계: 블록 분할 (필요한 경우)

*/

if (best->size > total_size + sizeof(struct bget_header) + BGET_MIN_SIZE) {

struct bget_header *new_block = NULL;

/* 새 자유 블록 생성 */

new_block = (struct bget_header *)((char *)best + total_size);

new_block->size = best->size - total_size;

new_block->magic = BGET_MAGIC_FREE;

new_block->next = best->next;

new_block->prev = best->prev;

/* 링크 업데이트 */

if (best->next)

best->next->prev = new_block;

if (best->prev)

best->prev->next = new_block;

/* 할당 블록 크기 조정 */
```

```
best->size = total_size;

} else {

/*

* 4단계: 전체 블록 사용 - 자유 리스트에서 제거

*/

if (best->next)

best->next->prev = best->prev;

if (best->prev)

best->prev->next = best->next;

}



/*

* 5단계: 할당 블록으로 표시

*/

best->magic = BGET_MAGIC_USED;

best->next = NULL;

best->prev = NULL;



/*

* 6단계: 사용자 데이터 영역 반환

*/

return (char *)best + sizeof(struct bget_header);

}
```

```
/*
* 메모리 블록 해제
*/
void bget_free(void *pool, void *ptr)
{
struct bget_header *h = NULL;
struct bget_header *pool_start = (struct bget_header *)pool;

if (!ptr)
return;

/*
* 1단계: 블록 헤더 획득
*/
h = (struct bget_header *)((char *)ptr - sizeof(struct bget_header));

/*
* 2단계: 매직 넘버 검증
*/
if (h->magic != BGET_MAGIC_USED) {
EMSG("Invalid magic in block %p: 0x%x", h, h->magic);
return;
```

```
}

/*
* 3단계: 자유 블록으로 표시
*/
h->magic = BGET_MAGIC_FREE;

/*
* 4단계: 인접 블록과 병합
*/
coalesce_free_blocks(pool_start, h);

/*
* 5단계: 자유 리스트에 추가
*/
insert_free_block(pool_start, h);
}

/*
* 인접한 자유 블록들을 병합
*/
static void coalesce_free_blocks(struct bget_header *pool_start,
struct bget_header *block)
```

```
{
struct bget_header *next_block = NULL;
struct bget_header *prev_block = NULL;

/*
* 다음 블록과 병합 시도
*/
next_block = (struct bget_header *)((char *)block + block->size);
if (is_valid_block(pool_start, next_block) &&
next_block->magic == BGET_MAGIC_FREE) {
/* 자유 리스트에서 제거 */
remove_from_free_list(next_block);
/* 크기 합산 */
block->size += next_block->size;
}

/*
* 이전 블록과 병합 시도
*/
prev_block = find_prev_block(pool_start, block);
if (prev_block && prev_block->magic == BGET_MAGIC_FREE &&
(char *)prev_block + prev_block->size == (char *)block) {
/* 자유 리스트에서 제거 */
```

```
remove_from_free_list(prev_block);

/* 크기 합산 */

prev_block->size += block->size;

/* 병합된 블록 사용 */

block = prev_block;

}

}
```

9. 보안 메커니즘 상세 구현

9.1 스택 카나리 보호 시스템

파일: `core/kernel/thread.c`

```
/*

* 스택 카나리 값들 - 런타임에 랜덤 생성

*/

static uint32_t start_canary_value;

static uint32_t end_canary_value;


/*

* 카나리 값 초기화

*/

void thread_init_canaries(void)
```

```
{
/*
* 하드웨어 랜덤 생성기 사용
*/
if (crypto_rng_read(&start_canary_value, sizeof(start_canary_value)) !=
TEE_SUCCESS) {
/* 폴백: 타이머 기반 시드 */
start_canary_value = read_cntpct() ^ 0xDEADBEEF;
}

if (crypto_rng_read(&end_canary_value, sizeof(end_canary_value)) !=
TEE_SUCCESS) {
end_canary_value = read_cntpct() ^ 0xCAFEBABE;
}

/* 카나리 값이 0이 되지 않도록 보장 */
if (!start_canary_value)
start_canary_value = 0xDEADBEEF;
if (!end_canary_value)
end_canary_value = 0xCAFEBABE;
}

/*
```

```
* 스택 카나리 검사 함수

*/

static void check_stack_canary(const char *name, size_t idx,

size_t stack_size, vaddr_t stack_end_va)

{

uint32_t *start_canary = NULL;

uint32_t *end_canary = NULL;

bool corruption_detected = false;



/*

* 1단계: 카나리 위치 계산

*/

start_canary = stack_end_va_to_start_canary(stack_size, stack_end_va);

end_canary = stack_end_va_to_end_canary(stack_size, stack_end_va);



/*

* 2단계: 시작 카나리 검사

*/

if (*start_canary != start_canary_value) {

EMSG("Stack corruption detected in %s[%zu] start canary", name, idx);

EMSG("Expected: 0x%x, Found: 0x%x", start_canary_value, *start_canary);

corruption_detected = true;

}
```

```
/*
* 3단계: 끝 카나리 검사
*/
if (*end_canary != end_canary_value) {
EMSG("Stack corruption detected in %s[%zu] end canary", name, idx);
EMSG("Expected: 0x%x, Found: 0x%x", end_canary_value, *end_canary);
corruption_detected = true;
}


/*
* 4단계: 부패 감지 시 시스템 패닉
*/
if (corruption_detected) {
EMSG("Stack corruption in %s[%zu], terminating", name, idx);
dump_stack_info(name, idx, stack_size, stack_end_va);
panic("Stack corruption detected");
}
}


/*
* 스택 정보 덤프 (디버깅용)
*/
```

```
static void dump_stack_info(const char *name, size_t idx,
size_t stack_size, vaddr_t stack_end_va)
{
vaddr_t stack_start = stack_end_va - stack_size;
uint32_t *ptr = NULL;
size_t i = 0;

EMSG("=== Stack Dump for %s[%zu] ===", name, idx);
EMSG("Stack range: 0x%lx - 0x%lx (size: %zu)",
stack_start, stack_end_va, stack_size);

/*
* 스택 내용 일부 덤프
*/
ptr = (uint32_t *)stack_start;
for (i = 0; i < 16 && (vaddr_t)ptr < stack_end_va; i++, ptr++) {
EMSG(" [%zu] 0x%lx: 0x%08x", i, (vaddr_t)ptr, *ptr);
}

EMSG("=== End Stack Dump ===");
}
```

9.2 메모리 태깅 (ARM MTE) 구현

```
/*
* ARM Memory Tagging Extension 지원
*/
#ifdef CFG_MEMTAG


/*
* 랜덤 태그로 메모리 영역 태깅
*/
void *memtag_set_random_tags(void *ptr, size_t size)
{
uintptr_t addr = (uintptr_t)ptr;
uintptr_t end_addr = addr + size;
uint8_t tag = 0;


/*
* 1단계: 16바이트 경계로 정렬
*/
addr = ROUNDDOWN(addr, MTE_GRANULE_SIZE);
end_addr = ROUNDUP(end_addr, MTE_GRANULE_SIZE);


/*
* 2단계: 각 그래뉼에 랜덤 태그 설정
```

```
*/
while (addr < end_addr) {
/* 랜덤 태그 생성 (0은 제외) */
do {
crypto_rng_read(&tag, sizeof(tag));
tag &= 0xF; /* 4비트 태그 */
} while (tag == 0);


/*
* 3단계: 메모리 태그 설정
*/
asm volatile("stg %0, [%1]" : : "r"(tag), "r"(addr));
addr += MTE_GRANULE_SIZE;
}


/*
* 4단계: 포인터에 태그 설정
*/
return memtag_set_ptr_tag(ptr, tag);
}


/*
* 메모리 태그 검증
```

```
*/

bool memtag_check_access(void *ptr, size_t size)

{

uintptr_t addr = (uintptr_t)ptr;

uintptr_t end_addr = addr + size;

uint8_t ptr_tag = memtag_get_ptr_tag(ptr);

uint8_t mem_tag = 0;



/*

* 각 그래뉼의 태그 검사

*/

while (addr < end_addr) {

asm volatile("ldg %0, [%1]" : "=r"(mem_tag) : "r"(addr));

if (mem_tag != ptr_tag) {

EMSG("Memory tag mismatch at 0x%lx: ptr_tag=%u, mem_tag=%u",

addr, ptr_tag, mem_tag);

return false;

}

addr += MTE_GRANULE_SIZE;

}



return true;

}
```

```
#endif /* CFG_MEMTAG */
```

10. 성능 최적화 메커니즘

10.1 캐시 관리 최적화

파일: `core/arch/arm/mm/core_mmu.c`

```
/*
* 캐시 일관성 유지를 위한 최적화된 함수들
*/

/*
* 데이터 캐시 플러시 (최적화된 버전)
*/
void dcache_clean_range_optimized(void *addr, size_t size)
{
uintptr_t start = (uintptr_t)addr;
uintptr_t end = start + size;
size_t cache_line_size = dcache_get_line_size();

/*
* 1단계: 캐시 라인 경계로 정렬
```

```
*/
start = ROUNDDOWN(start, cache_line_size);
end = ROUNDUP(end, cache_line_size);


/*
* 2단계: 배치 처리로 성능 최적화
*/
while (start < end) {
size_t batch_size = MIN(end - start, CACHE_BATCH_SIZE);
uintptr_t batch_end = start + batch_size;


/*
* 3단계: 배치 단위로 캐시 클린
*/
while (start < batch_end) {
asm volatile("dc cvac, %0" : : "r"(start));
start += cache_line_size;
}


/*
* 4단계: 메모리 배리어로 순서 보장
*/
dsb();
```

```
/*
* 5단계: CPU 양보 (긴 작업 시)
*/
if (batch_size >= CACHE_YIELD_THRESHOLD) {
cpu_relax();
}
}

/* 최종 동기화 */
dsb();
isb();
}

/*
* TLB 무효화 최적화
*/
void tlb_invalidate_range_optimized(vaddr_t va, size_t size)
{
vaddr_t end_va = va + size;
size_t page_size = SMALL_PAGE_SIZE;

/*
* 1단계: 범위 크기에 따른 전략 선택
```

```
*/
if (size >= TLB_FLUSH_ALL_THRESHOLD) {
/*
* 큰 범위: 전체 TLB 플러시가 더 효율적
*/
tlb_flush_all();
return;
}


/*
* 2단계: 페이지별 무효화
*/
va = ROUNDDOWN(va, page_size);
end_va = ROUNDUP(end_va, page_size);


while (va < end_va) {
/*
* 3단계: ASID 고려한 TLB 무효화
*/
asm volatile("tlbi vae1, %0" : : "r"(va >> 12));
va += page_size;
}
```

```
/*
* 4단계: 동기화
*/
dsb();
isb();
}
```

10.2 스핀락 최적화

```
/*
* 적응형 스핀락 구현
*/
struct adaptive_spinlock {
volatile uint32_t lock;
uint32_t spin_count;
uint32_t yield_threshold;
};

/*
* 적응형 스핀락 획득
*/
void adaptive_spin_lock(struct adaptive_spinlock *lock)
{
```

```
uint32_t spin_count = 0;

uint32_t backoff = 1;



while (1) {

/*

* 1단계: 락 상태 확인 (읽기 전용)

*/

while (READ_ONCE(lock->lock)) {

/*

* 2단계: 적응형 대기 전략

*/

if (spin_count < lock->yield_threshold) {

/*

* 짧은 스핀: CPU 사이클 절약

*/

for (uint32_t i = 0; i < backoff; i++) {

cpu_relax();

}

backoff = MIN(backoff * 2, MAX_BACKOFF);

} else {

/*

* 긴 대기: 스레드 양보

*/
```

```
thread_yield();

backoff = 1; /* 백오프 리셋 */

}

spin_count++;

}



/*

* 3단계: 원자적 락 획득 시도

*/

if (!__atomic_test_and_set(&lock->lock, __ATOMIC_ACQUIRE)) {

/*

* 4단계: 성공 시 통계 업데이트

*/

lock->spin_count = (lock->spin_count + spin_count) / 2;

/* 임계값 동적 조정 */

if (spin_count > lock->yield_threshold * 2) {

lock->yield_threshold = MIN(lock->yield_threshold * 2,

MAX_YIELD_THRESHOLD);

} else if (spin_count < lock->yield_threshold / 2) {

lock->yield_threshold = MAX(lock->yield_threshold / 2,

MIN_YIELD_THRESHOLD);

}

break;
```

```
}


/* 실패 시 다시 시도 */
spin_count++;
}
}


/*
* 스핀락 해제
*/
void adaptive_spin_unlock(struct adaptive_spinlock *lock)
{
__atomic_clear(&lock->lock, __ATOMIC_RELEASE);
}
```

이제 코드 흐름 상세 분석 문서가 크게 확장되었습니다. 다음으로 실습 예제 문서를 확장하겠습니다.